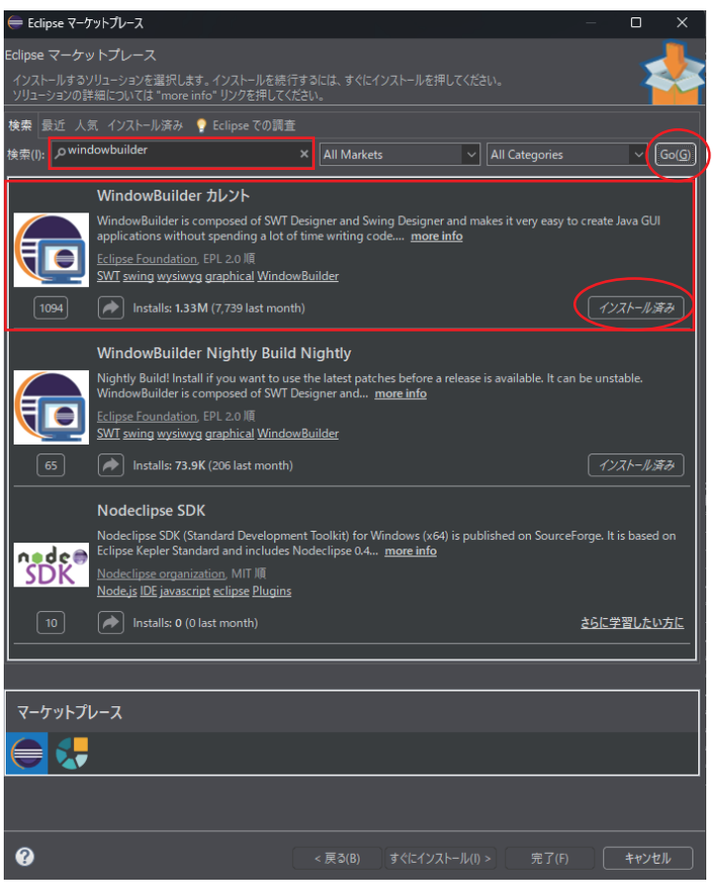
EclipseでJavaのフォームアプリケーションの作成方法

・Swing（スイング）＋ WindowBuilder
Java標準ライブラリ（一部を除く）に含まれており、最も情報が多い
昔ながらの仕組みなので、やや見た目が古い

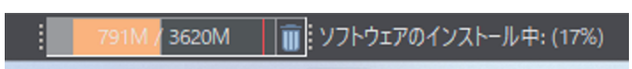
・JavaFX：e(fx)clipse(エフェックスクリプス)プラグイン ＋ Scene Builder
Swingより後のモダンな方法、環境構築の難度がやや高く、教材が少ない

Swing（スイング）＋ WindowBuilder　の環境構築

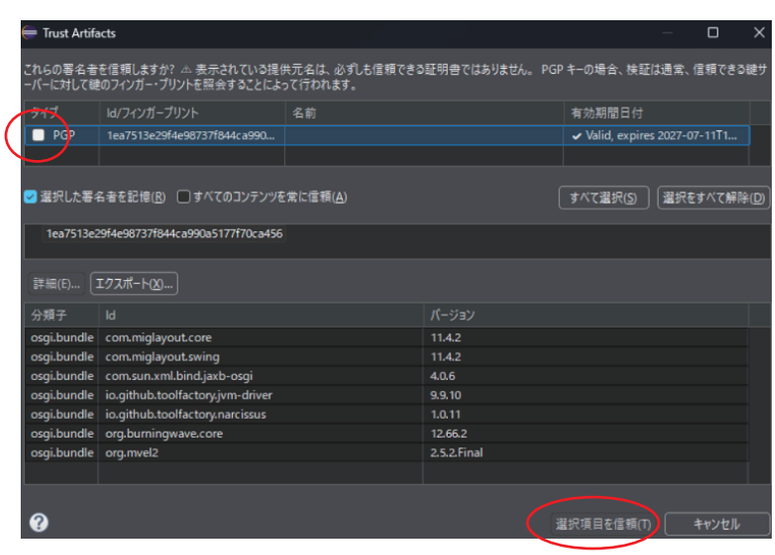
①ヒント ⇒ Eclipseマーケットプレイス ⇒「window builder」で検索



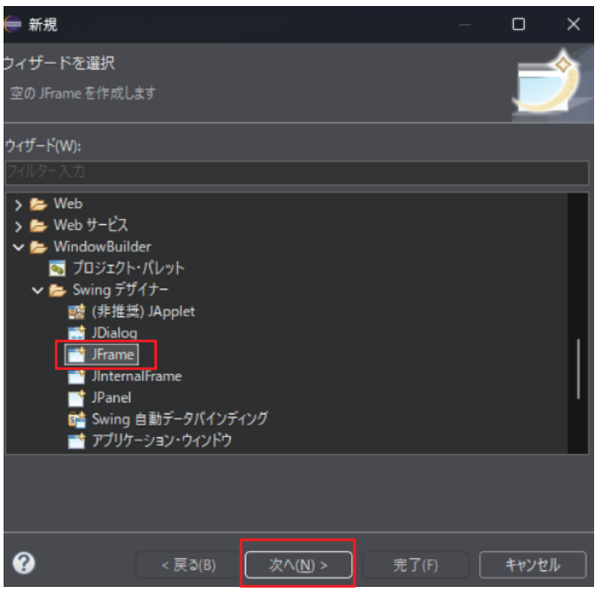
右下でインストール状況が把握できる



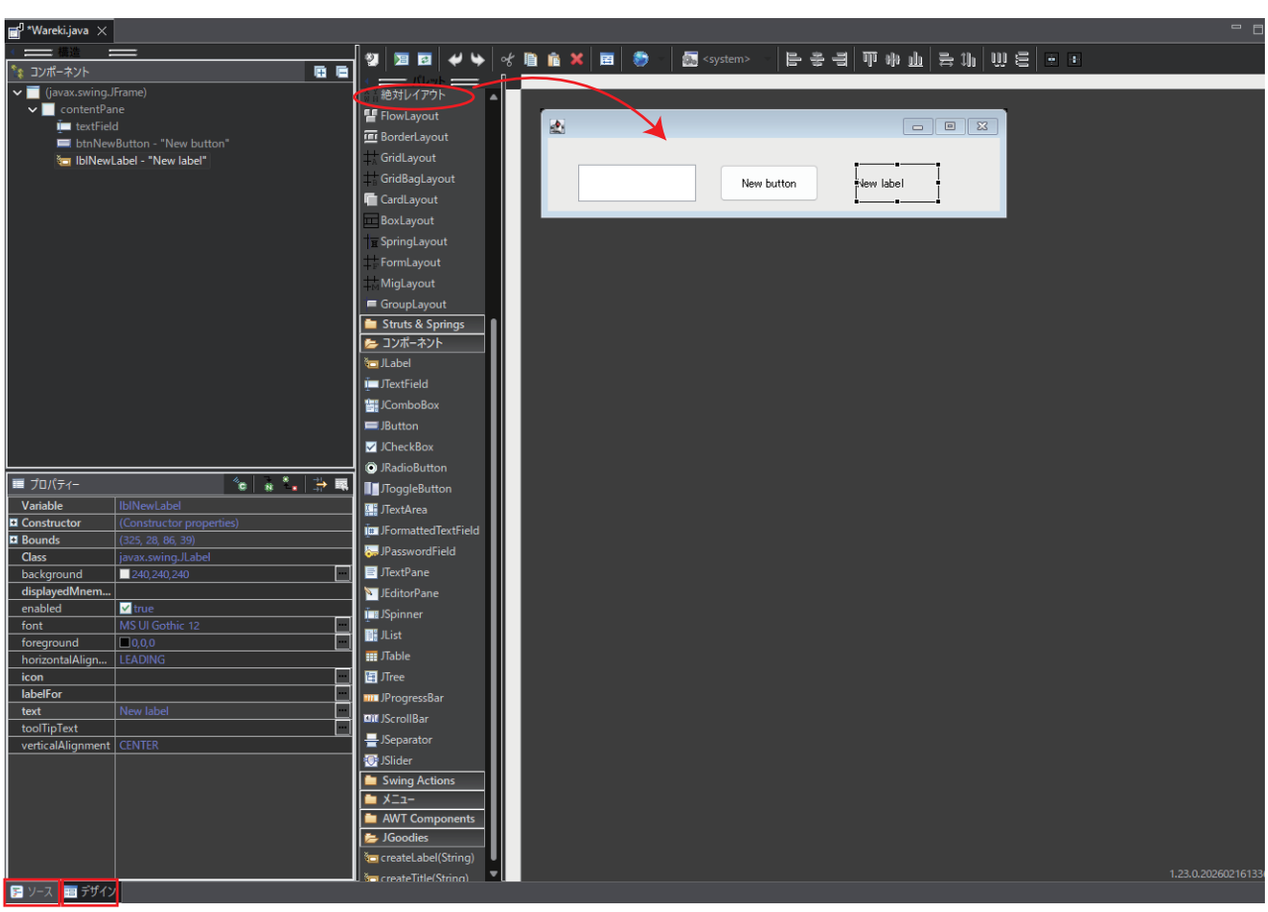
②データに含まれている署名にチェックを入れ「選択項目を信頼」

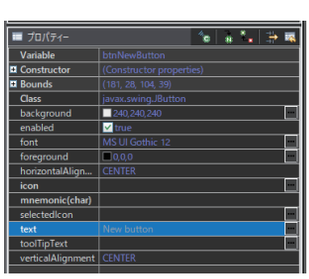


③プロジェクト「Form」を作成、
作成したFormプロジェクト上で新規 ⇒ その他 ⇒ WindowBuilder ⇒ Swingデザイナー ⇒ JFrame

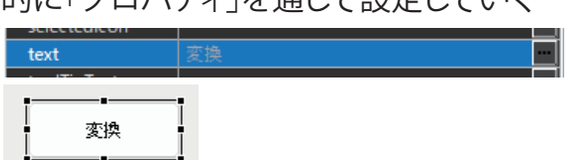


作成したプロジェクトは「ソース」コードと「デザイン」に分かれて作業する
※パレットの絶対レイアウトをウィンドウに適用すると形や配置を自由設計できるようになる

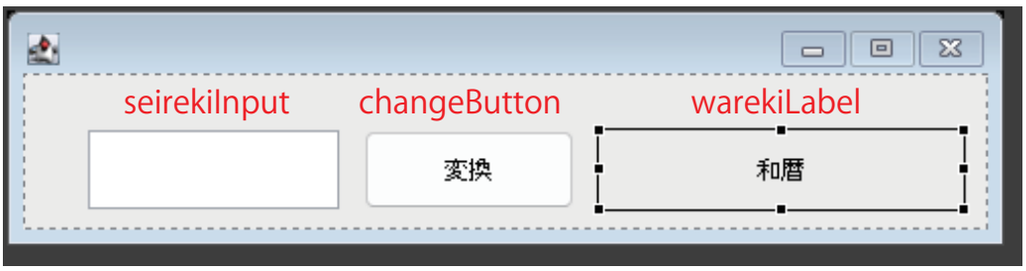




各オブジェクトは配置と大きさ以外は
基本的に「プロパティ」を通して設定していく



制作例　「テキストに西暦を入力して"変換"ボタンをおすと、和暦が出てくる」



Wareki.java

```java
import java.awt.EventQueue;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;
import java.util.HashMap;
import java.util.Map;

import javax.swing.JButton;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JPanel;
import javax.swing.JTextField;
import javax.swing.SwingConstants;

public class Wareki extends JFrame { //JFrameを継承したクラスとして生成される

    private static final long serialVersionUID = 1L; //JFrame保存⇒呼び込み時にバージョン番号指定が必要なのでとりあえず1

    // アプリの起動
    public static void main(String[] args) {
        EventQueue.invokeLater(new Runnable() { //実行スレッドに処理を登録
            public void run() {
                try {
                    Wareki frame = new Wareki(); //実体化
                    frame.setVisible(true); //ウィンドウ画面を表示
                } catch (Exception e) { //tryの中にトラブルが発生した時の情報を格納するエクセプション型
                    e.printStackTrace(); //エラー内容を追跡してコンソールに表示するメソッド
                }
            }
        });
    }

    //コンストラクタにて、辞書データの読み込みや、オブジェクトを実際に描画
    public Wareki() {

        //辞書データの読み込み準備
        Map<String, String> warekiMap = new HashMap<>(); //ディクショナリー型 warekiMapの生成
        try (BufferedReader br = new BufferedReader(new FileReader("wareki.csv"))) { //指定したファイルを読み変数brで参照
            String line; //読み込んだ文字列の格納用
            while ((line = br.readLine()) != null) { // 「line = br.readLine()」は一度実行の度に一行読み込んで変数lineに格納
                String[] data = line.split(","); // 内容をカンマを基準に分割して、配列dataに一時格納
                if (data.length == 2) { //ちゃんと「西暦」と「和暦」の2つの情報があればディクショナリー型に追加していく
                    warekiMap.put(data[0].trim(), data[1].trim()); //trim()は余計な空白や改行を取り除く
                }
            }
        } catch (IOException e) {
            e.printStackTrace();
        }

        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE); //×ボタンをおした際には、プログラムを完全に終了させるよう指示
        setBounds(100, 100, 492, 116); //アプリの初期位置とウィンドウの大きさ
        JPanel contentPane = new JPanel(); //オブジェクトを搭載するウィンドウ内の土台（以後ここにテキストやボタンをaddしていく）
//      contentPane.setBorder(new EmptyBorder(5, 5, 5, 5)); //見えない縁（paddingの役割）　自由レイアウトでは意味をなさない
        setContentPane(contentPane); //設定した土台を正式に採用
        contentPane.setLayout(null); //自動整列機能をOFF ※絶対座標を有効のままにしておく

        JTextField seirekiInput = new JTextField(); //オブジェクト名seirekiInputの初期化
        seirekiInput.setHorizontalAlignment(SwingConstants.CENTER);//プロパティ設定の反映
        seirekiInput.setBounds(32, 28, 124, 39); //位置の定義 左・上・横幅・高さ
        contentPane.add(seirekiInput); //設定した内容でウィンドウに描画

        JButton changeButton = new JButton("変換");//初期化
        changeButton.setBounds(168, 28, 104, 39);//位置の定義 左・上・横幅・高さ
        contentPane.add(changeButton);//設定した内容でウィンドウに描画

        JLabel warekiLabel = new JLabel("和暦");//初期化
        warekiLabel.setHorizontalAlignment(SwingConstants.CENTER);//プロパティ設定の反映
        warekiLabel.setBounds(284, 28, 180, 39);//位置の定義 左・上・横幅・高さ
        contentPane.add(warekiLabel);//設定した内容でウィンドウに描画

        //ボタンを押したときのイベント定義
        changeButton.addActionListener(new ActionListener() {
            public void actionPerformed(ActionEvent e) {
                String input = seirekiInput.getText(); // 入力された西暦を変数inputで参照
                String result = warekiMap.get(input); // 変数inputをキーにディクショナリー上で該当する値を変数resultで参照

                if (result != null) { //resultの中身が存在していれば
                    warekiLabel.setText(result);  //ラベルにセット
                } else {
                    warekiLabel.setText("データが見つかりません");
                }
            }
        });

    }

}
```

必要パッケージのインポート
（自動）

クラスの存在（自動）

メインメソッドにて
今回クラスの実体化と実際に描画
（自動）

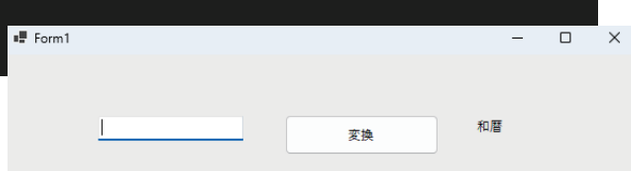
コンストラクタの存在（自動）

CSV ファイルの読み込み設計

描画されるオブジェクトの設定
とウィンドウパネルへのセット
（自動）
※実際に描画されるのは
メインメソッドから
呼び出されたコンストラクタにて

ボタンイベントの存在 (半自動)

ボタンイベントの中身を定義

参考資料：C#(.NET)で同様のアプリを作成した場合のコード

```
using System;
using System.Collections.Generic;
using System.IO;
using System.Windows.Forms;
using static System.Windows.Forms.VisualStyles.VisualStyleElement;

namespace wareki
{
    public partial class Form1 : Form
    {
        // 辞書データをフォーム全体で利用できるようにメンバー変数として宣言
        Dictionary<string, string> warekiMap = new Dictionary<string, string>();

        public Form1()
        {
            InitializeComponent();
            LoadWarekiData(); //辞書データを読み込むメソッドを加える
        }

        //.NETにおける辞書データ読み込みの仕組み
        void LoadWarekiData()
        {
            try
            {
                // wareki.csv ファイルのパス
                string filePath = Path.Combine(AppDomain.CurrentDomain.BaseDirectory, "wareki.csv");

                if (!File.Exists(filePath))
                {
                    MessageBox.Show("wareki.csv が見つかりません。", "エラー", MessageBoxButtons.OK, MessageBoxIcon.Error);
                    return;
                }

                // CSVファイルを一行ずつ読み込む
                using (StreamReader sr = new StreamReader(filePath))
                {
                    string line;
                    while ((line = sr.ReadLine()) != null)
                    {
                        string[] data = line.Split(',');

                        // 西暦と和暦の2つの情報があることを確認
                        if (data.Length == 2)
                        {
                            // 前後の空白をトリムして辞書に追加
                            warekiMap.Add(data[0].Trim(), data[1].Trim());
                        }
                    }
                }
            }
            catch (IOException ex)
            {
                MessageBox.Show($"ファイルの読み込み中にエラー発生: {ex.Message}", "エラー", MessageBoxButtons.OK,
                  MessageBoxIcon.Error);
                // エラーの詳細をコンソールに出力
                Console.WriteLine(ex.ToString());
            }
            catch (Exception ex)
            {
                MessageBox.Show($"予期せぬエラー発生: {ex.Message}", "エラー", MessageBoxButtons.OK, MessageBoxIcon.Error);
                Console.WriteLine(ex.ToString());
            }
        }

        //ボタンクリックイベント
        void changeButton_Click(object sender, EventArgs e)
        {
            // テキストボックス (seirekiInput) から西暦を取得し、
            // warekiMap を使って和暦に変換してラベル (warekiLabel) に表示する
            if (int.TryParse(seirekiInput.Text, out int seireki))
            {
                string seirekiString = seireki.ToString();
                if (warekiMap.ContainsKey(seirekiString))
                {
                    warekiLabel.Text = warekiMap[seirekiString];
                }
                else
                {
                    warekiLabel.Text = "対応する和暦なし";
                }
            }
            else
            {
                warekiLabel.Text = "有効な西暦を入力してください";
            }
        }
    }
}
```